

```

"""
=> text file ----> file.txt
=> binary files ----> file.bin or image,video,audio,.dat
=> csv file(comma seperated values) ==> file.csv ==> These are handle excel sheet type
data.
=====
when we handle file in python, we perform some operations :-
-----
1. write
2. read
3. create
4. delete
5. change directory

& ===== Handling text Files =====
syntax :-
-----

fileHandler = open("fileName","mode")
.....
fileHandler.close()

-----

filename => name.txt
mode :- The operation performed on file.

* there are some pre-defined modes for handle files :-
-----
w => write Mode => used to write data into the file.
NOTE :- It overWrite the existing Data.

a => append Mode => used to write data at last into the file.

r => read mode => Used to read file data.

x => create mode => Used to create the file.

w+ => write+read
r+ => read+write
a+ => append+read
"""
"""
for writing data into the file :-
-----
.write("text") :- Used to write plain text.
.writelines(list) => Write list data into the file.
"""

user_info=["Name = Ram and Age= 21\n","Name = Raman and Age= 22\n","Name = Ramesh and Age=
23\n"]

# lst=[]
# for i in range(1,101):
#     if(i%2==0):

```

```

#         lst.append(str(i))
#         lst.append("\n")

# file = open("textData.txt","r")

# file.write("This is first Line.\n")
# file.write("This is second Line.\n")
# file.write("This is third Line.\n")
# file.write("This is fourth Line.\n")
# file.write("This is fifth Line.\n")
# file.write("This is sixth Line.\n")

# file.writelines(user_info)
# file.seek(10)
# print(file.tell())
# data = file.read()
# print(data)
# file.close()

#* NOTE :- in "w and a" mode if file is not exists,then it create the file.

# ===== Reading Data from the text File =====
"""
Mode => r

There are three functions to read data from the file :-
-----
1 .read(n) :- By default ,It Read Entire file Data.But we can provide n to read n number
of characters.
2 .readline() :- Read first line from the file.
3 .readlines() :- Read all lines from the list and return in the form of list.
"""

# f = open("students records.txt","r")
# data=f.readlines()

# f.close()

#& ===== Binary File handline =====
# => Files with extensions :- .bin , .png , .jpg , .mp4 , .mp3, etc....

# NOTE :- pillow (pil) This Module is used to work with images.
# NOTE :- pickle This Module is used to work with binary files.

"""
Name of file :- fileName.bin

Modes for handle Binary File :-
-----
wb => write into the binary file.
ab => append into the binary file.
rb => read from the binary file.

```

```

x => create the binary file.

wb+ , rb+ , ab+ => read and write both operations.

To convert file data into binary format :-
-----
=> bytes(data) :- convert data into bytes.
"""

import pickle
# for write through pickle :- pickle.dump(data,fileHandler)
# for read through pickle :- pickle.load(filehandler)
"""

! The Python pickle module provides a way to serialize and deserialize Python object
structures.

*Serialization (Pickling):
-----
This is the process of converting a Python object hierarchy into a byte stream.
=> The resulting byte stream can be stored in a file, a database, or transmitted over a
network.
=> This allows for saving the state of a program or complex data structures for later use.

*Deserialization (Unpickling):
-----
=> This is the reverse process of converting a byte stream back into its original Python
object hierarchy.
=> It reconstructs the object with its original attributes, methods, and data.

Key Features and Considerations:
-----
Python-Specific:

=> pickle is specific to Python; pickled objects generally cannot be unpickled by other
programming languages.

Security Risk:
-----
=> Unpickling data from an untrusted source is a significant security risk.
=> Malicious pickle data can execute arbitrary code upon deserialization. Therefore, only
unpickle data from trusted sources.

Usage:
-----
=> pickle.dump(obj, file): Serializes obj and writes the resulting byte stream to the file
object.
=> pickle.load(file): Reads a pickled object from the file object and reconstructs it.
=> pickle.dumps(obj): Serializes obj and returns the resulting byte string.
=> pickle.loads(bytes_object): Deserializes bytes_object and returns the reconstructed
Python object.
"""

# lst=[10,21,32,46,65,76]
# dct={"name":"Ajay","Percentage":19,"Address":"Mathura"}

```

```

# file=open("binaryFile.bin","rb+")

# # file.write(bytes(lst))

# # pickle.dump(dct,file)

# data = pickle.load(file)
# for i in data:
#     print(data)

# file.close()

#& =====CSV(comma seperated values) file Handling =====
"""
CSV (Comma Separated Values) files are widely used to store tabular data.
In Python, handling CSV files is easy and efficient using the built-in csv module
or with libraries like Pandas for more advanced operations.

* We have to import csv module
import csv

* syntax for handling file :-
-----
fileHandler = open("fileName.csv","mode")
statements.....
fileHandler.close()

! MODES :- "r" , "w" , "a" , "x" , "r+" , "w+"

+++++
*for writing into the csv file :-
-----
We have to create a writer object.

writerObject = csv.writer(fileHandler)

=> for single row :- writerObject.writerow([])
=> for multiple rows :- writerObject.writerows([ [],[],[],[] ])

*for reading from the csv file :-
-----
We have to create a reader object

readerObject = csv.reader(fileHandler)

=>

"""
import csv

# file = open("Excel Data.csv","r")

```

```

#? This is a writer Object
# writer = csv.writer(file)

#~ for single row data.
# writer.writerow(["Name","Age","Address","Phone","Registration Id"])

#~ for multiple rows data.
# writer.writerows(
#     [
#         ["Ram",21,"Agra",379827323,101],
#         ["Ramesh",29,"mathura",489349834,102],
#         ["Suresh",26,"vrindavan",980658965,103],
#         ["Gagan",25,"raya",695806543,104],
#         ["Magan",22,"hathras",897594795,105],
#         ["Sunder",31,"delhi",9590485094,106],
#         ["Shyam",23,"noida",90865068565,107],
#         ["Manoj",22,"jaipur",9060950965,108]
#     ])

#? This is our reader object
"""
reader = csv.reader(file)

name=input("Enter name which you want to search data :- ")
for lines in reader:
    for data in lines:
        if(data.lower()==name.lower()):
            print(lines)
"""
# file.close()

"""
& ===== handling files using with statement =====
*By using with statement, we do not have to close the file manually.
*It is used to write or read data in an indentation(1 tab space)

syntax:-
-----
with open("fileName","mode") as fileHandler:
    statements
"""

# with open("Excel Data.csv","r") as file:
#     reader = csv.reader(file)

#     for data in reader:
#         print(data)

#&===== OS Module =====
import os
import shutil

```

```

"""
# This function is used to remove the non-empty directory(folder)
shutil.rmtree("c:\\Users\\sagar\\Desktop\\Rao html")
"""

# os.makedirs("operating system")
# os.removedirs("c:\\Users\\sagar\\Desktop\\raman")
# os.remove("c:\\Users\\sagar\\Desktop\\Rao html")
# os.rmdir("c:\\Users\\sagar\\Desktop\\Rao html")

# print(os.path.getctime("hello.txt"))

# =====
import time

# path = "basic programs.py"

# if (os.path.exists(path)):
#     creation_time = os.path.getctime(path)
#     readable_time = time.ctime(creation_time)
#     print(f"File was created on: {readable_time}")
# else:
#     print("File does not exist.")

# =====

# print(os.getcwd())
# os.chdir("c:\\Users\\sagar\\Desktop\\pythonWork")
# os.mkdir("Python Work")

# * return all files with their created time.
"""
all_files = os.listdir(path=".")

for file in all_files:
    date = os.path.getctime(file)
    readable_time = time.ctime(date)
    print(f"* {file} => {readable_time}")
    print(end="\n")

"""

# =====
# os.rename("textData.py", "textData.txt")

# !==== move one folder files into another folder files using shutil and os module ====

# source_path = "c:\\Users\\sagar\\Desktop\\Morning Python"
# destination_path = "c:\\Users\\sagar\\Desktop\\pythonWork"

# for files in os.listdir(source_path):
#     src_path = os.path.join(source_path, files)
#     dest_path = os.path.join(destination_path, files)
#     shutil.move(src_path, dest_path)

```

"""

📁 Directory & File Handling

Function	Description
os.getcwd()	Returns the current working directory
os.chdir(path)	Changes the current working directory
os.listdir(path='.')	Lists files and folders in a directory
os.mkdir(path)	Creates a single directory
os.makedirs(path)	Creates directories recursively
os.rmdir(path)	Removes a single directory
os.removedirs(path)	Removes directories recursively
os.remove(path)	Removes a file
os.rename(src, dst)	Renames a file or directory
os.path.exists(path)	Checks if a path exists
os.path.isfile(path)	Checks if it's a file
os.path.isdir(path)	Checks if it's a directory
os.path.join(a, b)	Joins paths (cross-platform safe)

"""